

## Data Structures and Algorithms

### Infix, Postfix and Prefix Expressions

#### LECTURE 4

Dr.Shada Mabgar

#### Algebraic Expression

- An algebraic expression is a legal combination of operands and the operators.
- Operand is the quantity (unit of data) on which a mathematical operation is performed.
- Operand may be a variable like  $x, y, z$  or a constant like  $5, 4, 0, 9, 1$  etc.
- Operator is a symbol which signifies a mathematical or logical operation between the operands.
- Example of familiar operators include  $+, -, *, /, ^$
- An example of expression as  $x + y * z$

## Introduction

- In **computers**, expressions with multiple parentheses can be inefficient. So, mathematicians have created different notations such as **infix**, **prefix**, and **postfix expressions** to reduce computational work.

العمل الحسابي

Data Structure Dr. Shada Mabgar

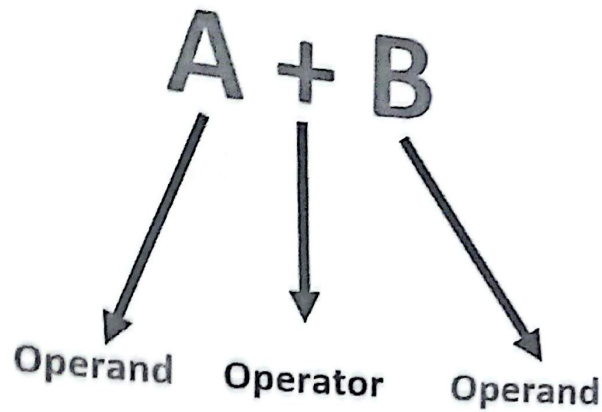
## Infix, Postfix and Prefix notations

- The way to write **arithmetic** expression is known as a **notation**.
- An arithmetic expression can be written in three different but **equivalent** notations :

1. Infix Notation
2. Prefix Notation
3. Postfix Notation

Data Structure Dr. Shada Mabgar

## Infix Expressions



Data Structure Dr. Shada Mabgar

## Infix Expressions

- We write expression in infix notation:

e.g. :

$$a - b + c$$

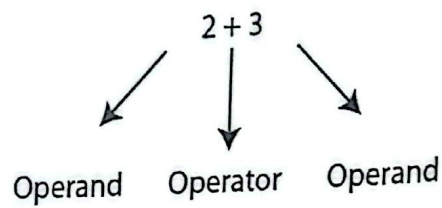
- Where operators are used **in-between operands**.
- It is easy for us humans to read, write, and speak in infix notation but the same does not go well with computing devices.
- An algorithm to process infix notation could be **difficult and costly** in terms of time and space **consumption**.

Data Structure Dr. Shada Mabgar

## Infix Expression

- We can take a simple addition sum:

$$2 + 3$$



Data Structure Dr.shada Mabgar

## Infix Expressions

- We can see that the **Operator** is sandwiched in between the Operands for the Infix notation.
- This is true for any mathematical formulae which uses Infix notation:

**Example :**

Prefix Expressions

$$2 + 3$$

$$5 - 2$$

and

$$2 + 3 + 4$$

$$2 + 5 / 2$$

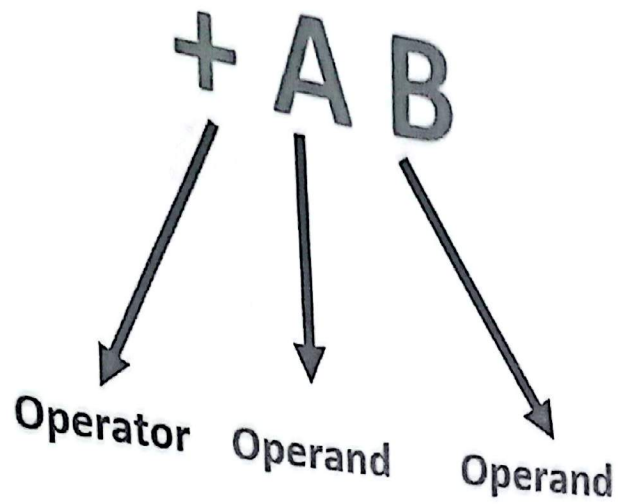
$$1 + 2 / 3$$

- When we mix Operands we have a **precedence** (that is, order of operations) **issue**.

Data Structure Dr.shada Mabgar



## Prefix Expressions



Data Structure Drshada Mabgar

## Prefix Expressions

- Puts the **operator** before all of the **operand**.
- For example,  $+ab$ .
- This is equivalent to its infix notation  $a + b$ .

The Prefix Example of :  $2+3$

**Infix:**  $2 + 3$

**Prefix:**  $+ 2 3$

Data Structure Drshada Mabgar

## Prefix Notation

Example :

1- Infix:  $2 + 3 + 4$

Prefix:  $++234$

2- Infix:  $A+B+C+D \Rightarrow +AB+C+D \Rightarrow ++ABC+D$

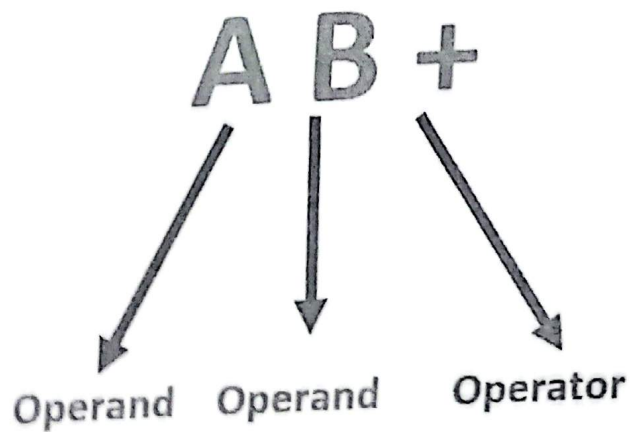
$\Rightarrow$  Prefix:  $+++ABCD$

3- Infix:  $A*B+C/D \Rightarrow *AB+C/D \Rightarrow +*AB+/CD$

$\Rightarrow$  Prefix:  $+*AB/CD$

Data Structure Drshada Mabgar

## Postfix Expressions



Data Structure Drshada Mabgar

## Postfix Notation

- Puts the **operator** after all of the **operands**.
- This is just like the **prefix** notation, but the **operand** comes at the end of the expression, and similarly removes the need for brackets within any expressions.

**Example :**

**Infix:**  $2 + 3$

**Postfix:**  $2\ 3\ +$

Data Structure Dr. Shada Mabgar

## Postfix Notation

**Example :**

**1- Infix:**  $2 + 3 + 4$

**Postfix:**  $23+4+$

**2- Infix:**  $A+B+C+D \Rightarrow AB+C+D \Rightarrow ABC++D \Rightarrow ABCD+++$

**X Postfix:**  $AB+C+D+$

**3- Infix:**  $A*B+C/D \Rightarrow AB*+C/D \Rightarrow AB*+C.D/ \Rightarrow AB*CD/+$

**Postfix:**  $AB*CD/+$

Data Structure Dr. Shada Mabgar

## Precedence Rules

قواعد أسبقية العمليات  
Operator precedence rules follow:

- **Parentheses:** expressions inside parentheses are evaluated first  
الأقواس أولاً
- **Exponentiation:** exponents are evaluated next  
الأسس ثانياً
- **Multiplication and division:** multiplication and division are evaluated before addition and subtraction  
الضرب أو القسمة ثالثاً
- **Addition and subtraction:** finally, addition and subtraction are evaluated last  
الجمع والطرح
- If an **expression** has **multiple** operators with the same precedence, the evaluation of those operators occurs from **left to right**.

١٥

Data Structure Drshada Mabgar

## Precedence

- As multiplication operation has precedence over addition,  $b * c$  will be evaluated first.

$$a + b * c \Rightarrow a + (b * c)$$

١٦

Data Structure Drshada Mabgar



## Associativity

- Describes the rule where operators with the same precedence appear in an expression.
- For example, in expression  $a + b - c$ ,
- both  $+$  and  $-$  have the same precedence, then which part of the expression will be evaluated first, is determined by associativity of those operators.
- Here, both  $+$  and  $-$  are left associative, so the expression will be evaluated as  $(a + b) - c$ .

11

Data Structure Dr. Shada Mabgar

## Table of Operator Precedence

| Operators                   | Symbols      |
|-----------------------------|--------------|
| Parenthesis                 | $() [] \{\}$ |
| Exponents                   | $^$          |
| Multiplication and Division | $*, /$       |
| Addition and Subtraction    | $+, -$       |

Data Structure Dr. Shada Mabgar

## Examples of Infix, Prefix, and Postfix

| Infix Expression | Prefix Expression | Postfix Expression |
|------------------|-------------------|--------------------|
| $A+B*C+D$        | $++A*BCD$         | $ABC*+D+$          |
| $(A+B)*(C+D)$    | $*+AB+CD$         | $AB+CD*+$          |
| $A*B+C*D$        | $+*AB*CD$         | $AB*CD*+$          |
| $A+B+C+D$        | $+++ABCD$         | $AB+C+D+$          |

١١

Data Structure Dr. Shada Mabgar

## Convert an Infix expression to a Postfix expression by using stack

1. Scan the infix expression from **left to right**.
2. If the scanned character is an **operand**, put it in the postfix expression.
3. Otherwise, do the following :
  - If the **precedence** of the current scanned operator is higher than the precedence of the operator **on top of the stack**, or if the **stack is empty**, or if the stack contains a '**(**', then push the current operator onto the stack.
  - Else, pop all operators from the stack that have precedence higher than or equal to that of the current operator. After that push the current operator onto the stack.
4. If the scanned character is a '**(**', push it to the stack.
5. If the scanned character is a '**)**', pop the stack and output it until a '**(**' is encountered, and discard both the parenthesis.
6. **Repeat steps 2-5** until the infix expression is scanned.
7. Once the scanning is over, Pop the stack and add the operators in the postfix expression until it is not empty.
8. Finally, print the postfix expression.

١٢

Data Structure Dr. Shada Mabgar

## Convert into Postfix

Convert  $2*3/(2-1)+5*3$  into Postfix form

| Expression | Stack     | Output     |
|------------|-----------|------------|
| 2          | Empty     |            |
| *          | 2         | 2          |
| 3          | 2         | 23         |
| /          | 23        | 23*        |
| (          | 23*       | 23*        |
| 2          | 23*       | 23*        |
| -          | 23*2      | 23*2       |
| 1          | 23*2      | 23*2       |
| )          | 23*21     | 23*21      |
| +          | 23*21-    | 23*21-     |
| 5          | 23*21-    | 23*21-     |
| *          | 23*21-5   | 23*21-5    |
| 3          | 23*21-53  | 23*21-53   |
|            | 23*21-53* | 23*21-53*+ |

Data Structure Dr. Shada Mabgar

## Evaluation a Postfix Expression

- Each **operator** in a postfix string refers to the previous **two operands** in the string.
- Suppose that each time we read an **operand** we push it into a stack.
- When we reach an **operator**, its **operands** will then be top **two** elements on the stack
- We can then **pop** these two elements, perform the indicated operation on them, and **push** the result on the stack.
- So that it will be available for use as an **operand** of the next **operator**.

Data Structure Dr. Shada Mabgar



## Evaluating Postfix Notation

- Use a **stack** to evaluate an expression in **postfix** notation.
- The **postfix** expression to be evaluated is **scanned** from **left to right**.
- **Variables** or **constants** are **pushed** onto the **stack**.
- When an operator is **encountered**, the indicated action is performed using the **top elements** of the **stack**, and the result replaces the **operands** on the **stack**.

17

Data Structures for Java Edition

## Evaluating a Postfix Expression

- 1- Initialise an empty stack
- 2- While token remain in the input stream
  - Read next token
  - If token is a number, push it into the stack
  - Else, if token is an operator, pop top two tokens off the stack, apply the operator, and push the answer back into the stack
- 3- Pop the answer off the stack .

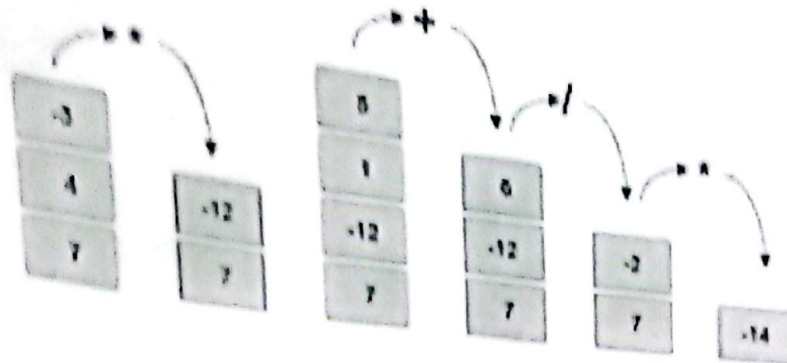
18

Data Structures for Java Edition



# Evaluating Postfix Expressions

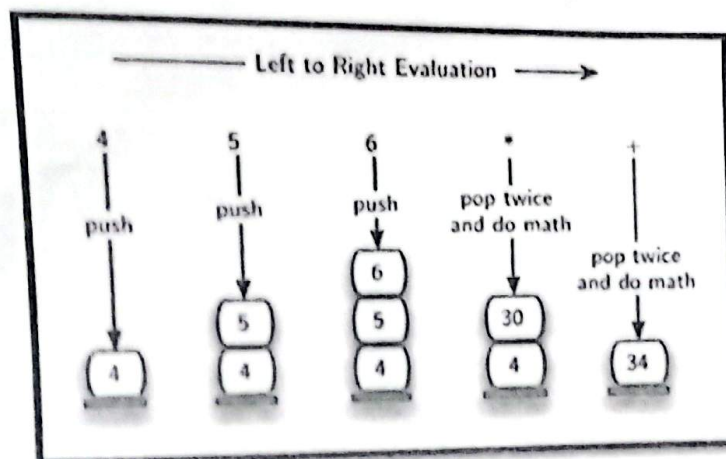
• Expression = 7 4 -3 \* 1 5 + / \*



Data Structure Dr. Shada Malgar

## Evaluating a Postfix Expression

Expression: 456\*+



Data Structure Dr. Shada Malgar

## Evaluating a Postfix Expression

| Step | Input Symbol | Operation                     | Stack | Calculation |
|------|--------------|-------------------------------|-------|-------------|
| 1.   | 4            | Push                          | 4     |             |
| 2.   | 5            | Push                          | 4,5   |             |
| 3.   | 6            | Push                          | 4,5,6 |             |
| 4.   | *            | Pop(2 elements)<br>& Evaluate | 4     | $5*6=30$    |
| 5.   |              | Push result(30)               | 4,30  |             |
| 6.   | +            | Pop(2 elements)<br>& Evaluate | Empty | $4+30=34$   |
| 7.   |              | Push result(34)               | 34    |             |
| 8.   |              | No more elements(pop)         | Empty | 34(Result)  |